



Unit Test

M226A

Mars Climate Orbiter (1999)

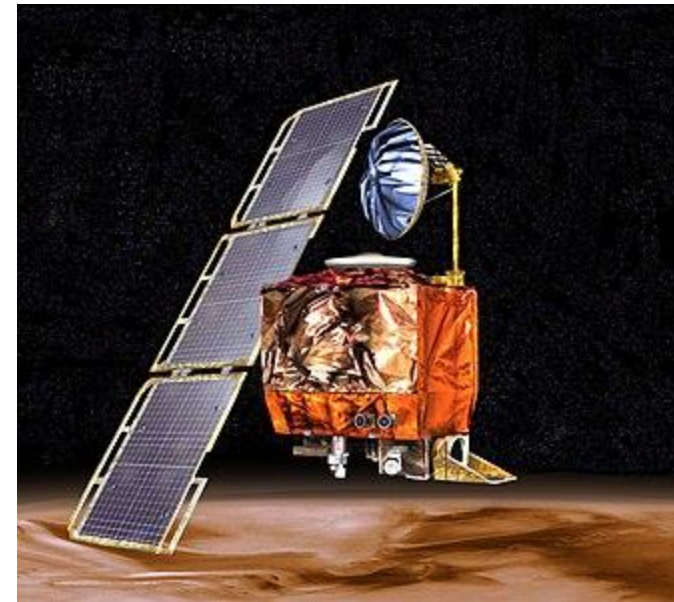
Il caso Mars Climate Orbiter: Un errore da 327 milioni di dollari

La missione era semplice: inviare una sonda su Marte per studiarne il clima. Tuttavia, dopo mesi di viaggio, la sonda si è letteralmente disintegrata nell'atmosfera marziana invece di entrare in orbita.

Cosa è successo?

Il problema risiedeva in due "unità di codice" (moduli software) che dovevano comunicare tra loro:

- **Modulo per i tecnici a terra (scritto da Lockheed Martin):** Calcolava la spinta dei motori usando il sistema imperiale britannico (**Libbre-forza per secondo**).
- **Modulo B (scritto dalla NASA):** Riceveva quei dati e si aspettava che fossero nel sistema metrico decimale (**Newton per secondo**).



Caso Mariner 1 (1962)

"Il trattino più costoso della storia"

Il bug non fu un errore di logica complessa, ma un errore di **trascrizione** dalla specifica cartacea al codice sorgente.

- **La Specifica:** La formula richiedeva la velocità media, indicata matematicamente con una barra sopra il simbolo R. La barra istruiva il sistema a ignorare le piccole fluttuazioni casuali del segnale radar.
- **Il Codice:** La barra (l'overbar) venne omessa. Il software iniziò a elaborare ogni singola variazione del segnale come se fosse un cambiamento reale di direzione.
- **Il Risultato:** Il computer di bordo cercò di correggere fluttuazioni inesistenti, portando il razzo fuori rotta. La missione fu abortita e il razzo distrutto dopo soli 4 minuti e 53 secondi dal lancio.
- **Costo missione:** milioni di dollari

Caso Therac

- Il Therac-25 era una macchina per la radioterapia controllata interamente da un computer. Il bug principale era una **Race Condition** (una "condizione di corsa" tra processi).
- **Il Problema:** La macchina aveva due modalità: "X-ray" (alta potenza, con uno schermo protettivo) e "Electron" (bassa potenza, senza schermo).
- **Il Bug:** Se l'operatore premereva i tasti per impostare la modalità X-ray e poi, molto velocemente (in meno di 8 secondi), correggeva un errore di battitura tornando su Electron, il software si "confondeva".
- **La Conseguenza:** La macchina attivava il raggio ad altissima potenza (X-ray) ma **senza** inserire lo schermo protettivo. I pazienti ricevevano una dose di radiazioni 100 volte superiore a quella letale, senza che i medici se ne accorgessero subito.

Introduzione

Il **test unitario** è una metodologia di verifica del software che analizza singole unità di codice sorgente per accertarne il corretto funzionamento.

I suoi principali vantaggi includono:

- **Rilevamento precoce dei bug:** l'esecuzione frequente dei test permette di individuare e risolvere le anomalie già nelle prime fasi del ciclo di sviluppo.
- **Refactoring sicuro:** i test garantiscono che il software continui a comportarsi come previsto anche dopo modifiche strutturali o ottimizzazioni del codice.
- **Innalzamento della qualità:** assicurando che ogni componente sia privo di errori logici, si ottiene un prodotto finale più robusto, pulito e manutenibile.

Introduzione

Una pratica fondamentale per ogni sviluppatore consiste nell'eseguire test unitari sistematici. Questi garantiscono che ogni singolo componente rispetti i requisiti definiti e favoriscono un'integrazione del software rapida e di alta qualità.

Un po' di storia

Anni '50 - '60

Il concetto di testare singole "unità" di un programma esiste da quando esiste la programmazione modulare.

- **John von Neumann** e i pionieri dell'informatica degli anni '50 già applicavano forme di verifica sui singoli algoritmi prima di integrarli.
- Documenti tecnici degli anni '60 (legati ad esempio ai sistemi della NASA) citano già l'importanza di testare i singoli componenti in isolamento.

Kent Beck

Se oggi parliamo di "Unit Test" come pratica standard, lo dobbiamo principalmente a **Kent Beck**. È lui che ha formalizzato e automatizzato il processo negli anni '90:

- **SUnit (1989)**: Mentre lavorava con il linguaggio Smalltalk, Beck scrisse il primo vero framework dedicato ai test unitari, chiamato SUnit.
- **JUnit (1997)**: Durante un volo da Zurigo ad Atlanta, Kent Beck ed **Erich Gamma** (uno degli autori del celebre libro sui *Design Patterns*) scrissero la prima versione di JUnit per Java.
- **Extreme Programming (XP) e TDD**: Beck ha reso lo unit testing uno dei pilastri della metodologia *Extreme Programming* e ha "riscoperto" (o meglio, formalizzato) il **Test-Driven Development (TDD)**, la pratica di scrivere il test *prima* del codice.

Test Unitari

Un test unitario è una procedura volta a verificare che una singola unità di codice (tipicamente un metodo) rispetti le specifiche logiche per cui è stata progettata. In Java, l'obiettivo dello Unit Testing è convalidare il comportamento di ogni metodo, confrontando il risultato ottenuto con il valore atteso in diversi scenari.

Esempio

Ipotizziamo la situazione in cui abbiamo creato una funzione che restituisce il valore assoluto di un numero.

Un caso di test (**test case**) verificherà che, quando passiamo il valore -5 al metodo, esso restituirà 5 come valore di ritorno.

Se il test ha esito negativo evidentemente c'è un errore da correggere.

JUnit

JUnit è un framework per facilitare l'implementazione di programmi di test in Java.

Di seguito sono elencati alcuni principi del framework JUnit:

- Ciascuno unit test è un metodo
- Ogni metodo di test fornito è trovato ed eseguito dal framework.
- Si hanno una varietà di asserzioni per automatizzare il controllo dei risultati dei test
- Integrazione con tool e IDE diffusi (Eclipse, IntelliJ, VS Code)

Configurazione JUnit

Configurare Proxy IntelliJ

<https://www.jetbrains.com/help/idea/junit.html#intellij>

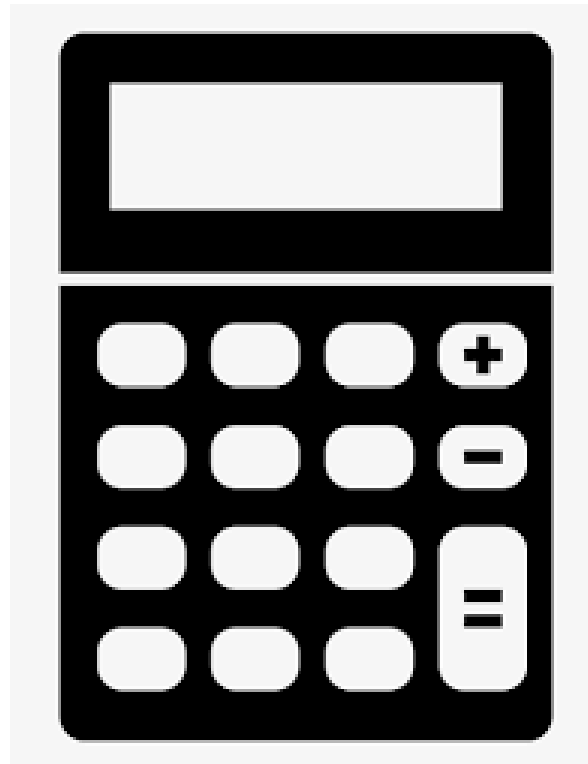
Fondamenti

Per verificare se il codice si comporta come ci si aspetta si usa una assertion, ovvero una chiamata al metodo `assert`, che verifica se il risultato ottenuto (actual) coincide con il risultato atteso (expected)

- La classe che fornisce i metodi usati per valutare le esecuzioni è la classe `Assert`
- `assertTrue(boolean condition)` valuta se `condition` è `true`, se non è così il test ha trovato un errore.
- `assertEquals` verifica se due valori sono uguali.
- I metodi `assert` registrano fallimenti o errori e li riportano
- Quando si verifica un fallimento o un errore, l'esecuzione del metodo di test viene interrotta, ma verranno eseguiti gli altri metodi di test della stessa classe

Esempio guidato

Supponiamo di avere una classe `Calculator` con un metodo che permette di calcolare la somma di due numeri.



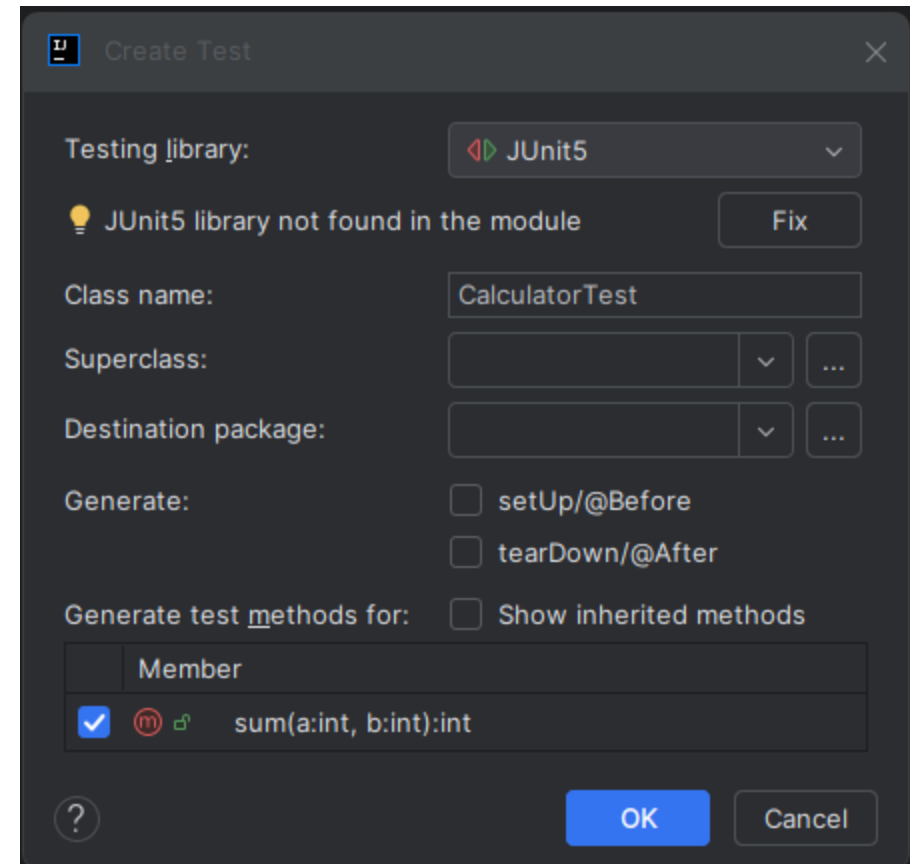
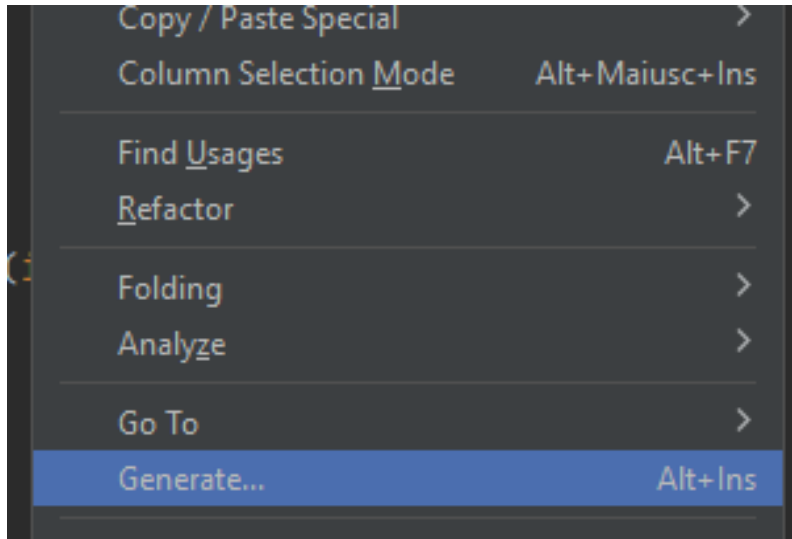
Classe Calculator

- Creare la classe Calculator.
- Creare il metodo sum(int a, int b).

```
public class Calcolatrice {  
    public int sum(int a, int b) {  
        return a + b;  
    }  
}
```

Creare i Tests

1. Per generare i test effettuare un click destro sulla classe Calculator e selezionare test
2. Selezionare sum(): int



Implementare un test

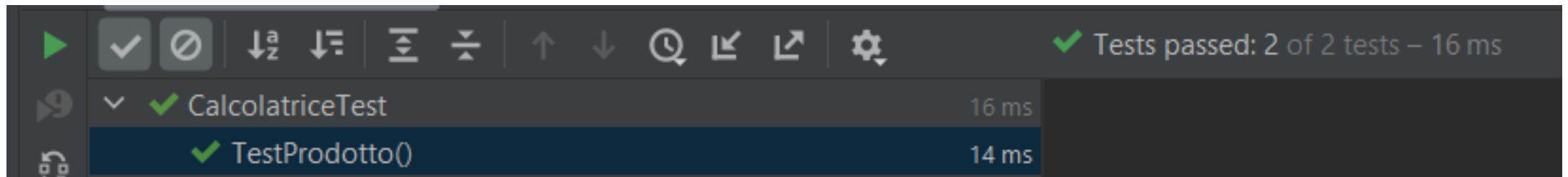
```
class CalculatorTest {  
  
    @Test  
    void shouldReturnCorrectSumWhenTwoPositiveIntegersAreAdded() {  
        // 1. Arrange: Configurazione dei dati di input  
        Calculator calc = new Calculator();  
        int firstOperand = 5;  
        int secondOperand = 10;  
        int expectedResult = 15;  
  
        // 2. Act: Esecuzione del calcolo  
        int actualResult = calc.sum(firstOperand, secondOperand);  
  
        // 3. Assert: Verifica della correttezza del risultato  
        assertEquals(expectedResult, actualResult, message: "La somma di 5 e 10 dovrebbe essere 15");  
    }  
}
```

assertEquals

Il metodo **assertEquals** confronta il valore restituito dall'esecuzione (**actual**) con il risultato atteso (**expected**).

In caso di mancata corrispondenza, l'asserzione solleva una **failure**, segnalando l'errore con un messaggio descrittivo che evidenzia la discrepanza.

Avviare il test



Esempio 2

Nel secondo esempio è rappresentato un test su una classe che descrive un carrello della spesa.



Item

Preparazione della
classe Item che
servirà alla classe
ShoppingCart

```
public class Item { 12 usages
    private String name; 5 usages
    private double prize; 5 usages

    public Item(String name, double prize) {...}
    public String getNome() { return name; } no usages
    public double getPrezzo() { return prize; } 1 usage

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return false;
        Item item = (Item) o;
        return Double.compare(prize, item.prize) == 0 && Objects.equals(name, item.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, prize);
    }
}
```

Shopping Cart

Implementazione della classe ShoppingCart.

Nel test andremo a verificare il metodo getTotalPrice.

```
public class ShoppingCart { 2 usages
    private List<Item> items; 4 usages

    public ShoppingCart() { 1 usage
        this.items = new ArrayList<>();
    }

    public List<Item> getItems() { 1 usage
        return items;
    }

    public void addItem(Item item) { 3 usages
        if (item != null) {
            items.add(item);
        }
    }

    public double getTotalPrice() { 1 usage
        double total = 0.0;

        for (Item i : items) {
            total = total + i.getPrezzo();
        }
        return total;
    }
}
```

BeforeEach

L'annotation ***BeforeEach*** permette di preparare i test con le inizializzazioni del caso (istanziare un'oggetto ShoppingCart)

```
class ShoppingCartTest {  
    private ShoppingCart cart; 4 usages  
    @BeforeEach  
    void setUp() {  
        cart = new ShoppingCart();  
    }  
}
```

DisplayName

L'annotation **DisplayName** permette di far apparire la descrizione nel test durante l'esecuzione.

```
@Test
@DisplayName("Should return the correct sum of all items added to the cart")
void shouldSumAllItemPricesCorrectly() {
    // ARRANGE
    Item bread = new Item("Bread", 1.50);
```


Nome dei metodi di test (Convenzione)

L'obiettivo di un nome come ***shouldSumAllItemPricesCorrectly*** è rendere il test auto-esplicativo. La convenzione si basa su una struttura ricorrente:

1. Il Prefisso (Should)

Perché: Trasforma il test in una frase di senso compiuto.

Concetto: Definisce un obbligo del software. Se il test fallisce, la frase "Il sistema dovrebbe fare X" diventa falsa, rendendo immediata la comprensione del bug.

2. L'Azione o Scenario (SumAllItemPrices)

Perché: Identifica chiaramente l'unità logica sotto esame.

Concetto: Non descrive il nome del metodo tecnico, ma la funzionalità dal punto di vista dell'utente o del sistema.

3. Il Risultato Atteso (Correctly / When...)

Perché: Specifica la condizione di successo o il contesto.

Concetto: Un test non deve solo "eseguire" del codice, deve validare un esito specifico.

AAA

Il modello **AAA (Arrange-Act-Assert)** rappresenta ormai lo standard di riferimento per la scrittura dei test unitari. Questo schema prevede la suddivisione del test in tre fasi distinte, ognuna con una responsabilità specifica:

- **Arrange (Preparazione):** In questa fase si predispone l'ambiente necessario al test. Qui si inizializzano gli oggetti, si configurano i mock e si definiscono i parametri di input.
- **Act (Esecuzione):** Rappresenta il cuore del test e consiste nell'invocazione effettiva del metodo o della funzionalità sotto esame.
- **Assert (Verifica):** È la fase conclusiva in cui si valida il risultato ottenuto, verificando che l'output o lo stato del sistema corrispondano alle aspettative definite.

Questo approccio garantisce una struttura chiara, facilitando la lettura e la manutenzione del codice di test.

```
class ShoppingCartTest {  
    private ShoppingCart cart; 4 usages  
    @BeforeEach  
    void setUp() {  
        cart = new ShoppingCart();  
    }  
    @Test  
    @DisplayName("Should return the correct sum of all items added to the cart")  
    void shouldSumAllItemPricesCorrectly() {  
        // ARRANGE  
        Item bread = new Item(name: "Bread", prize: 1.50);  
        Item milk = new Item(name: "Milk", prize: 2.20);  
        double expectedTotal = 3.70;  
        // ACT  
        cart.addItem(bread);  
        cart.addItem(milk);  
        double actualTotal = cart.getTotalPrice();  
        // ASSERT  
        assertEquals(expectedTotal, actualTotal, delta: 0.0001, message: "The total price calculation failed.");  
    }  
}
```

Con valori double si
utilizzare il delta per
verificare il margine di
errore dal valore atteso.

shouldAddItemToCollectionWhenItemIsValid()

```
@Test
void shouldAddItemToCollectionWhenItemIsValid() {
    Item validItem = new Item( name: "Laptop", prize: 1200.0);

    cart.addItem(validItem);

    assertTrue(cart.getItems().contains(validItem), message: "Il carrello dovrebbe contenere l'item aggiunto");
    assertEquals( expected: 1, cart.getItems().size(), message: "Il carrello dovrebbe avere esattamente 1 elemento");
}
```

Esempio 3 – Difficile da testare

```
public class DiscountService {  
    // Dipende direttamente dall'orologio di sistema (LocalDateTime.now())  
    public double getDiscount() {  
        LocalDateTime now = LocalDateTime.now();  
        if (now.getDayOfWeek() == DayOfWeek.SUNDAY) {  
            return 0.20; // 20% di sconto la domenica  
        }  
        return 0.05; // 5% negli altri giorni  
    }  
}
```

Esempio 3 - Testabile

```
public class DiscountService {  
    // Passiamo la data come parametro (Dependency Injection)  
    public double calculateDiscount(LocalDate dateTime) {  
        if (dateTime.getDayOfWeek() == DayOfWeek.SUNDAY) {  
            return 0.20;  
        }  
        return 0.05;  
    }  
}
```

Esempio 3 - Testabile

```
public class DiscountServiceTest {  
    @Test  
    public void testGetDiscountOnSunday() {  
        DiscountService service = new DiscountService();  
        // "Simuliamo" una domenica  
        LocalDateTime sunday = LocalDateTime.of(2023, Month.OCTOBER, 22, 10, 0);  
        double discount = service.calculateDiscount(sunday);  
        assertEquals(0.20, discount, "La domenica lo sconto deve essere del 20%");  
    }  
    @Test  
    public void testGetDiscountOnMonday() {  
        DiscountService service = new DiscountService();  
        // "Simuliamo" un lunedì  
        LocalDateTime monday = LocalDateTime.of(2023, Month.OCTOBER, 23, 10, 0);  
        double discount = service.calculateDiscount(monday);  
        assertEquals(0.05, discount, "Il lunedì lo sconto deve essere del 5%");  
    }  
}
```